Práctica de laboratorio: Trayecto Trenes

Autor: David Benavides. **Revisores:** . **Última modificación:** 07/03/2026|

1. Objetivo

El objetivo de este ejercicio es diseñar un modelo de clases para representar **trayectos de tren**, así como mostrar cómo una **interfaz puede actuar como contrato de diseño**, permitiendo diferentes implementaciones internas de un mismo tipo abstracto.

Para ello se pide:

1. Definir una **interfaz `TrayectoTren`** que especifique los métodos que debe ofrecer cualquier trayecto de tren.
2. Implementar dicha interfaz mediante **dos clases distintas**, que utilicen estructuras de datos diferentes para almacenar las paradas del trayecto.

Las dos implementaciones que se deben desarrollar son:

- **`TrayectoTrenList`**: almacena las paradas utilizando una colección `List`.
- **`TrayectoTrenSet`**: almacena las paradas utilizando una colección `Set`.

Ambas clases deben **implementar la interfaz `TrayectoTren`** y proporcionar el mismo comportamiento externo.

La información que se quiere representar y gestionar es como la que se ve en la imagen a continuación.

TRAYECTO SEVILLA-MADRID - AV City (Nº 02471)

ESTACIÓN	LLEGADA	SALIDA
SEVILLA-SANTA JUSTA		**07.00**
CÓRDOBA	07.45	07.50
VILLANUEVA DE CÓRDOBA-LOS PEDROCHES	08.13	08.14
PUERTOLLANO	08.40	08.41
CIUDAD REAL	08.57	08.58
MADRID-PUERTA DE ATOCHA	**10.02**	

Duración del trayecto entre SEVILLA-SANTA JUSTA y MADRID-PUERTA DE ATOCHA: 3 h. y 2 min.

2. Modelo del dominio

Un **trayecto de tren** está compuesto por una secuencia ordenada de **paradas**.

Cada parada contiene:

- el **nombre de la estación**
- la **hora de llegada del tren**
- la **hora de salida del tren**

Para representar esta información se definirá el tipo **`Parada`** (decidir si hacerlo con un record o una clase).

La primera parada representa la **estación de origen** y la última la **estación de destino**.

3. Tipo enumerado `TipoTren`

Se define el tipo enumerado:

`TipoTren`
con los siguientes valores posibles:

```
AV_CITY
AVE
AVE_LD
LD_MD
ALVIA
INTERCITY
```

4. Interfaz `TrayectoTren`

Se debe definir una **interfaz `TrayectoTren`** que represente el comportamiento común de cualquier trayecto de tren. Esta interfaz debe declarar al menos los siguientes métodos y las clases que la implementen deben poder almacenar las siguientes propiedades.

Propiedades consultables

Un objeto de tipo `TrayectoTren` debe proporcionar acceso a:

- **Código del tren** (`String`)
- **Nombre del trayecto** (`String`)
- **Tipo de tren** (`TipoTren`)
- **Conjunto de paradas del trayecto**
- **Hora de salida del trayecto**
- **Hora de llegada del trayecto**
- **Duración total del trayecto**

La duración se calcula como la diferencia entre la hora de salida de la primera parada y la hora de llegada a la última.

`Operaciones`

La interfaz debe declarar las siguientes operaciones:

Obtener hora de salida de una estación

`LocalTime getHoraSalida(String estacion)`
Devuelve la hora a la que el tren **sale de la estación indicada**.

Debe devolver `null` si:

- la estación no pertenece al trayecto
- la estación es la última del trayecto

Obtener hora de llegada de una estación

`LocalTime getHoraLlegada(String estacion)`
Devuelve la hora a la que el tren **llega a la estación indicada**.

Debe devolver `null` si:

- la estación no pertenece al trayecto
- la estación es la primera del trayecto

Añadir estación intermedia

```
void anadirEstacionIntermedia(int posicion,
                              String estacion,
                              LocalTime horaLlegada,
                              LocalTime horaSalida)
```

Añade una nueva parada intermedia al trayecto en la posición indicada.

Se lanzará `IllegalArgumentException` en los siguientes casos:

1. Si la posición no es intermedia (debe ser mayor o igual que 1 y menor que el número de paradas).
2. Si la hora de salida es anterior a la hora de llegada.
3. Si la hora de llegada es anterior a la hora de salida de la parada anterior.
4. Si la hora de salida es posterior a la hora de llegada de la parada siguiente.

Eliminar estación intermedia

`void eliminarEstacionIntermedia(String estacion)`
Elimina la parada correspondiente a la estación indicada.

Debe lanzar `IllegalArgumentException` si:

- la estación es la primera del trayecto
- la estación es la última del trayecto
- la estación no pertenece al trayecto

5. Constructores

Cada implementación concreta de `TrayectoTren` debe proporcionar un constructor con los siguientes parámetros:

- código del tren
- nombre del trayecto
- tipo de tren
- estación origen
- estación destino
- hora de salida del origen
- hora de llegada al destino

Este constructor creará un trayecto formado inicialmente por **dos paradas**:

- parada de origen
- parada de destino

Para la parada de origen:

- hora de llegada = `null`
- hora de salida = hora de salida indicada

Para la parada de destino:

- hora de llegada = hora de llegada indicada
- hora de salida = `null`

6. Restricciones

Deben cumplirse las siguientes restricciones:

R1: El código del tren debe estar formado por **exactamente 5 dígitos**.

R2: La hora de salida de la estación de origen **no puede ser nula**.

R3: La hora de llegada a la estación de destino **no puede ser nula**.

R4: La hora de salida de la estación de origen debe ser **anterior** a la hora de llegada a la estación de destino.

7. Representación como cadena

el nombre del trayecto, seguido de un guion, el tipo de tren, seguido del código del tren entre paréntesis, y después, por cada estación, el nombre de la estación, un tabulador, la hora de llegada a la estación, un tabulador y la hora de salida; para la de origen la hora de llegada se dejará en blanco, así como la hora de salida en la de destino. Por ejemplo:

```
Sevilla-Madrid-AV_CITY (02471)
      SEVILLA-SANTA JUSTA                    07:00
      CORDOBA                    07:45       07:50
      ...
      MADRID-PUERTA DE ATOCHA    10:02
```

8. Igualdad y orden

Igualdad

Dos objetos `TrayectoTren` se consideran iguales si tienen:

- el mismo **nombre de trayecto**
- la misma **hora de salida**
- el mismo **código de tren**

Orden natural

Los trayectos deben ser comparables y ordenarse según:

1. nombre del trayecto
2. hora de salida
3. código del tren

9. Implementaciones requeridas

Se deben implementar **dos clases concretas** que implementen la interfaz `TrayectoTren`:

`TrayectoTrenImplList`

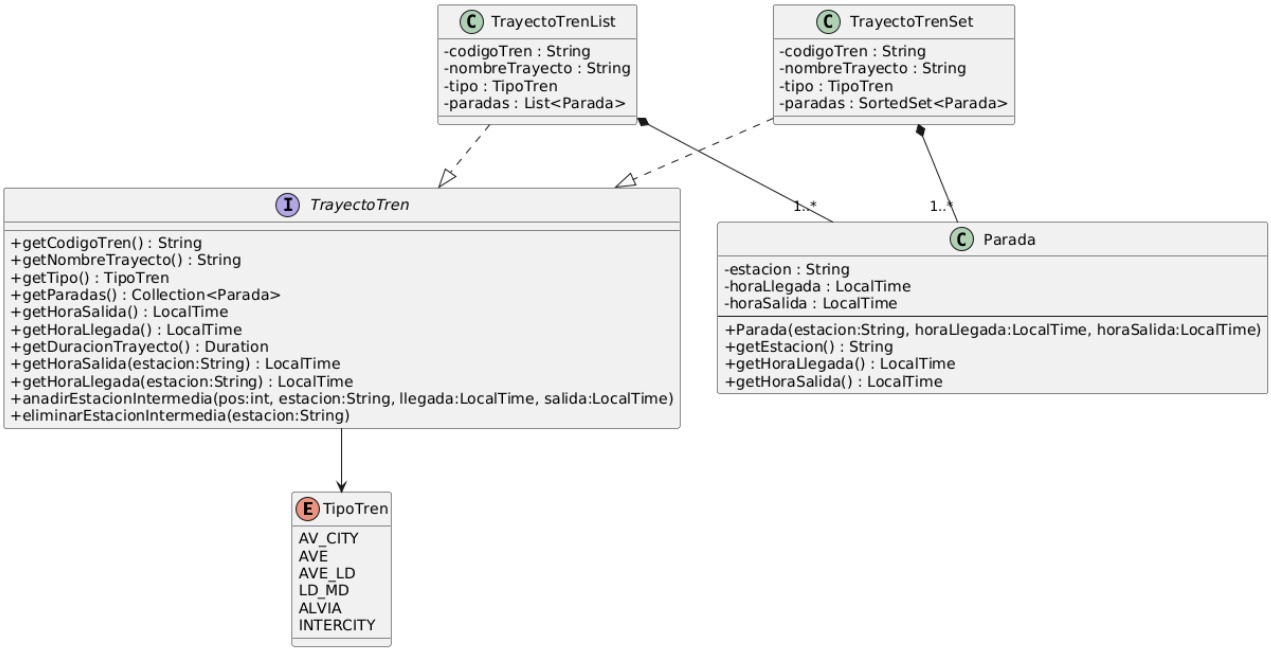
- Debe almacenar las paradas en una colección `List<Parada>`.
- El orden de la lista representa el orden del recorrido del tren.

`TrayectoTrenImplSet`

- Debe almacenar las paradas en una colección `Set<Parada>`.
- Se debe garantizar el **orden del recorrido** utilizando una estructura adecuada (`LinkedHashSet` o `TreeSet`).

10. Diagrama de clases esperado

El diseño del sistema debe incluir al menos las siguientes clases y tipos:


TrayectoTrenList
-codigoTren : String
-nombreTrayecto : String
-tipo : TipoTren
-paradas : List<Parada>
TrayectoTrenSet
-codigoTren : String
-nombreTrayecto : String
-tipo : TipoTren
-paradas : SortedSet<Parada>
TrayectoTren
+getCodigoTren() : String
+getNombreTrayecto() : String
+getTipo() : TipoTren
+getParadas() : Collection<Parada>
+getHoraSalida() : LocalTime
+getHoraLlegada() : LocalTime
+getDuracionTrayecto() : Duration
+getHoraSalida(estacion:String) : LocalTime
+getHoraLlegada(estacion:String) : LocalTime
+anadirEstacionIntermedia(pos:int, estacion:String, llegada:LocalTime, salida:LocalTime)
+eliminarEstacionIntermedia(estacion:String)
1..*
1..*
Parada
-estacion : String
-horaLlegada : LocalTime
-horaSalida : LocalTime
+Parada(estacion:String, horaLlegada:LocalTime, horaSalida:LocalTime)
+getEstacion() : String
+getHoraLlegada() : LocalTime
+getHoraSalida() : LocalTime
TipoTren
AV_CITY
AVE
AVE_LD
LD_MD
ALVIA
INTERCITY